

Multi-Layer Perceptrons (MLPs)

1 Intro

- **Perceptron** = single-layer network → can only solve simple, linearly separable problems.
 - **MLP (Multi-Layer Perceptron)** = multiple layers of neurons → can solve **non-linear** problems.
 - Core idea: Stack layers + add **non-linear activations**.
-

2 The Problem

- A single perceptron fails on complex patterns (e.g., XOR).
 - To overcome this, we need networks that can:
 1. Combine features in flexible ways.
 2. Learn curves, circles, and other non-linear boundaries.
-

3 Perceptron with Sigmoid

- Instead of the **step function** (only 0/1), use a **sigmoid function**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- **Benefits:**
 1. Smooth, differentiable (good for gradient descent).
 2. Outputs a probability between 0 and 1.
 - This change allows us to train perceptrons using **calculus-based optimization**.
-

4 Adding Nodes in Hidden Layer

- Adding a **hidden layer** means stacking multiple perceptrons together.
 - Hidden neurons can detect intermediate patterns:
 1. First layer: basic shapes/features.
 2. Next layers: combinations of those features.
 - This is where the **power of MLPs** comes from — hierarchical learning.
-

5 Adding Nodes in Input

- More input nodes → handle more features.
 - Example:
 1. 2 inputs → (x, y).
-

2. 3 inputs $\rightarrow (x, y, z)$.
- Each input connects to all hidden layer neurons with different weights.
- More inputs = richer data, but also higher complexity.
-

6 Adding Nodes in Output

- Output layer size depends on the task:
 1. **Binary classification** $\rightarrow$ 1 output neuron (probability 0–1).
 2. **Multi-class classification** $\rightarrow$ multiple neurons (one per class, often with softmax).
 3. **Regression** $\rightarrow$ 1 neuron (predicts a continuous number).
-

7 Deep Neural Network

- An MLP with **multiple hidden layers** = **Deep Neural Network (DNN)**.
 - More layers $\rightarrow$ can represent very complex functions.
 - Downside: harder to train (vanishing gradients, overfitting).
 - But with modern tricks (ReLU, batch norm, dropout, GPUs), DNNs are trainable.
-

8 TF Playground Demo

Click -> [TensorFlow Playground](#)

- A great interactive tool to visualize MLPs.
 - You can adjust:
 1. **of hidden layers.**
 2. **of neurons.**
 3. Activation functions.
 - It shows how decision boundaries evolve as training progresses.
-

9 Outro

- MLPs are the foundation of **deep learning**.
 - They fix perceptron's weakness by stacking layers + using non-linear activations.
 - MLPs can model **almost any function** if given enough neurons and layers (Universal Approximation Theorem).
-

Quick Recap

- **MLP Structure:** Input $\rightarrow$ Hidden layer(s) $\rightarrow$ Output.
 - **Why better than perceptron?** Can solve **non-linear problems**.
 - **Key ingredients:**
-

1. Non-linear activations (sigmoid, ReLU, tanh).
 2. Multiple hidden layers.
 3. Backpropagation for training.
- **Leads to:** Deep Neural Networks (DNNs).
-